

ENTERPRISE JAVA • COMPUTER SCIENCE & ENGINEERING

# Advanced Java Architecture and Enterprise Technologies



An Exhaustive Examination Report — Presentation Deck

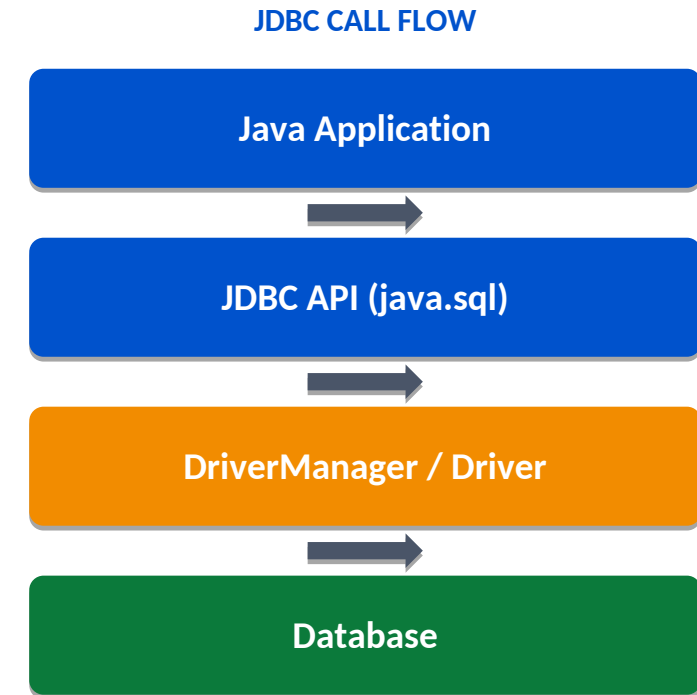
JDBC   Remote Method Invocation (RMI)   Java Servlets   Applets & AWT

# Introduction to Enterprise Java

- ▶ **Academic Focus:** Transitioning from basic programming to enterprise-grade, distributed architectures.
- ▶ **Core Prerequisites (Unit 2-4):**
  - Packages and Interfaces.
  - Exception Handling (try, catch, finally).
  - Multithreading — crucial for Servlet performance.
  - Applets & AWT — client-side foundations.
- ▶ **Unit 5 Core Pillars:**
  - **Java Database Connectivity (JDBC)**
  - **Remote Method Invocation (RMI)**
  - **Java Servlets**

# Java Database Connectivity (JDBC) Architecture

- ▶ **Definition:** An industry-standard API (java.sql) letting Java apps interact seamlessly with relational databases.
- ▶ **Key Benefit:** Database-agnostic code — a driver translates Java API calls into the database's proprietary language.
- ▶ **Architectural Models:**
  - **2-Tier:** Client talks directly to the database. Fast, but less secure and harder to scale.
  - **3-Tier:** Client talks to a middle-tier app server, which talks to the database. Secure & scalable — the web standard.



# The Four Types of JDBC Drivers

## Type 1 JDBC-ODBC Bridge

Translates JDBC to ODBC. Obsolete — severe performance overhead and manual client installs.

## Type 2 Native-API (Partial Java)

Translates JDBC into native C/C++ DB libraries. Faster than Type 1, but needs platform-specific client installs.

## Type 3 Network Protocol (Middleware)

Pure Java driver talking to a middleware server. Highly portable; great for many different databases.

## Type 4 Pure Java — Thin Driver

The modern standard. 100% Java, direct over sockets. Highest performance, zero client install.

# The 5 Steps of JDBC Execution

**1**

## Register the Driver Class

Load the driver into JVM memory dynamically using `Class.forName()`.

**2**

## Establish the Connection

Open a channel via `DriverManager.getConnection()` with a JDBC URL and credentials.

**3**

## Create the Statement Object

Prepare a vehicle for the SQL query using `con.createStatement()`.

**4**

## Execute the SQL Query

Dispatch with `executeQuery()` (SELECT) or `executeUpdate()` (INSERT/UPDATE/DELETE).

**5**

## Close the Resources

Prevent memory leaks by explicitly invoking `.close()` on connections and statements.

# Basic JDBC Program Example

BasicJDBCSelect.java

```
import java.sql.*;
public class BasicJDBCSelect {
    public static void main(String[] args) {
        try {
            // Step 1: Register the Driver Class
            Class.forName("com.mysql.cj.jdbc.Driver");
            // Step 2: Establish the Connection
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/university_db", "root", "password");
            // Step 3: Create the Statement Object
            Statement stmt = con.createStatement();
            // Step 4: Execute the SQL Query
            ResultSet rs = stmt.executeQuery("SELECT * FROM students");
            // Process the Result (iterate the cursor row by row)
            while (rs.next()) {
                System.out.println("ID: " + rs.getInt(1) + " Name: " + rs.getString(2));
            }
            // Step 5: Close the Resources
            con.close();
        } catch (Exception e) {
            System.out.println(e);    // ClassNotFoundException / SQLException
        }
    }
}
```

# Statement vs. PreparedStatement

## Statement

- ▶ Executes static SQL, compiled dynamically at runtime.
- ▶ Extremely vulnerable to SQL Injection attacks.

## PreparedStatement

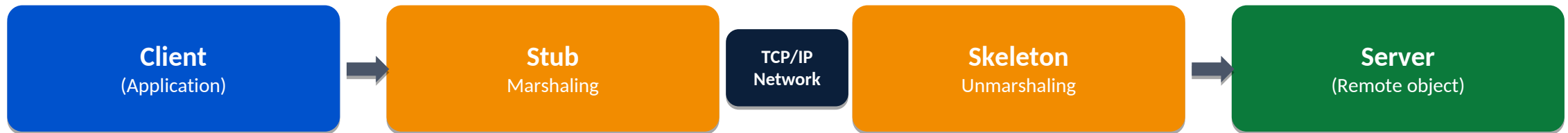
- ▶ Precompiled SQL using ? placeholders for variables.
- ▶ Adv 1: Neutralizes SQL injection — input treated as data, not code.
- ▶ Adv 2: Massive performance gains when the same query runs in a loop.

## CallableStatement

- ▶ Inherits from PreparedStatement.
- ▶ Exclusively used to execute complex stored procedures on the DB server.

# Remote Method Invocation (RMI) Architecture

- ▶ **Purpose:** Lets a Java object in one JVM invoke methods on an object in a separate, remote JVM across a network.
- ▶ **The Proxy Layer (crucial for exams):**
  - **Stub (client-side):** local proxy that performs Marshaling — packaging method name & parameters into a network byte stream.
  - **Skeleton (server-side):** receives the stream, performs Unmarshaling, and executes the real method on the server.



Marshaling → byte stream over the wire → Unmarshaling → real method executes; return value flows back the same path.



# RMI Application Lifecycle

**Serialization:** Any object passed over the network in RMI must implement `java.io.Serializable`.



1

## Define Interface

Must extend `Remote` and throw `RemoteException`.

2

## Implement Logic

Typically extends `UnicastRemoteObject`.

3

## Start Registry

Start the localized Java DNS via `rmiregistry`.

4

## Bind Server

Instantiate the remote object and bind it to a string identifier in the registry.

5

## Client Lookup

Client queries the registry identifier, retrieves the Stub, and invokes methods.

# Simple Client/Server with RMI — Part 1

## 1 • Define the Remote Interface — Hello.java

```
import java.rmi.Remote;  
import java.rmi.RemoteException;  
  
// Remote interface for the distributed application  
public interface Hello extends Remote {  
    void printMsg() throws RemoteException;  
}
```

## 2 • Implement the Interface — ImplExample.java

```
// Implementing the remote interface  
public class ImplExample implements Hello {  
    // Body for the remote method  
    public void printMsg() {  
        System.out.println(  
            "This is an example RMI program executing on the  
Server.");  
    }  
}
```

### Key rules:

- ▶ The interface must extend **java.rmi.Remote** and every method must declare **throws RemoteException**.
- ▶ Any object passed across JVMs must be **Serializable**.

# Simple Client/Server with RMI — Part 2

## 3 • Server — bind the object • Server.java

```
import java.rmi.registry.Registry;
import java.rmi.registry.LocateRegistry;
import java.rmi.server.UnicastRemoteObject;

public class Server extends ImplExample {
    public static void main(String[] args) {
        try {
            ImplExample obj = new ImplExample();
            Hello stub = (Hello)
                UnicastRemoteObject.exportObject(obj, 0);
            Registry registry = LocateRegistry.getRegistry();
            registry.bind("Hello", stub);
            System.err.println("RMI Server running...");
        } catch (Exception e) { e.printStackTrace(); }
    }
}
```

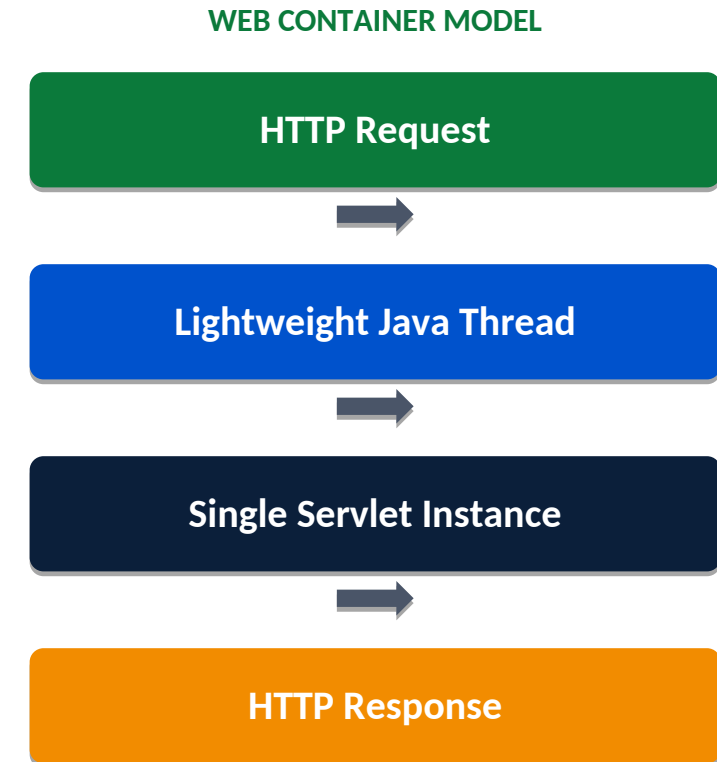
## 4 • Client — lookup & invoke • Client.java

```
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class Client {
    public static void main(String[] args) {
        try {
            Registry registry =
                LocateRegistry.getRegistry(null);
            Hello stub = (Hello) registry.lookup("Hello");
            stub.printMsg();
            System.out.println(
                "Remote method invoked from the client.");
        } catch (Exception e) { e.printStackTrace(); }
    }
}
```

# Java Servlets: Web Application Foundation

- ▶ **Purpose:** Java classes that run inside a Web Container (e.g., Tomcat) to generate dynamic HTML, manage sessions, and reach databases via JDBC.
- ▶ **Servlets vs. CGI:**
  - **CGI flaw:** spawns a heavy OS process per HTTP request — servers crash under load.
  - **Servlet advantage:** one instance in memory + a lightweight Java thread per request — massively scalable.



# Core Servlet Packages & Hierarchy

## javax.servlet

**Protocol-independent.**

- ▶ Base Servlet interface
- ▶ Abstract GenericServlet class

## javax.servlet.http

**HTTP-specific.**

- ▶ Contains the HttpServlet class

### Why extend HttpServlet?

- ▶ Provides a sophisticated internal **service()** method that automatically analyzes the incoming HTTP request.
- ▶ Seamlessly routes the request to specialized handlers like **doGet()** or **doPost()** — keeping code clean.

# The Deterministic Servlet Life Cycle

Orchestrated entirely by the Web Container:



# Basic Servlet Implementation Example

```
LifeCycleExampleServlet.java
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
public class LifeCycleExampleServlet extends HttpServlet {
    // final -> thread-safe under concurrent requests
    private final String greetingMessage = "Welcome to Advanced Java Servlets!";
    @Override public void init() throws ServletException {
        System.out.println("Servlet Initialization Phase Initiated.");
    }
    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {
        resp.setContentType("text/html");
        try (PrintWriter out = resp.getWriter()) {
            out.println("<html><body><h1>" + greetingMessage + "</h1>");
            out.println("<p>The Servlet container routed this GET request.</p>");
            out.println("</body></html>");
        }
    }
    @Override public void destroy() {
        System.out.println("Servlet Destroyed. Releasing resources.");
    }
}
```

# Handling HTTP Requests: doGet() vs doPost()

| Architectural Feature | doGet() · HTTP GET                     | doPost() · HTTP POST                   |
|-----------------------|----------------------------------------|----------------------------------------|
| Primary Purpose       | Retrieve, fetch, or view data.         | Send, save, or update data.            |
| Data Visibility       | URL-encoded (visible in address bar).  | Hidden inside the message body.        |
| Security Posture      | Insecure for passwords / keys.         | Secure for sensitive transactions.     |
| Payload Limits        | Restricted (max URL length).           | Theoretically limitless capacity.      |
| Idempotency           | Strictly idempotent (safe to refresh). | Non-idempotent (refresh duplicates).   |
| Caching               | Can be cached and bookmarked.          | Cannot be bookmarked; caching blocked. |



# Managing State & Configuration

**HTTP is a stateless protocol.** Servlets manage state & configuration via two core interfaces:

## ServletConfig

Per-Servlet Scope

- ▶ Created during the `init()` phase.
- ▶ Completely private to that individual Servlet.
- ▶ Injects localized params (e.g. a file path or cryptographic salt).

## ServletContext

Application-Wide Scope

- ▶ Represents the global environment of the web app.
- ▶ Shared by all Servlets in the application.
- ▶ Inter-servlet communication via `setAttribute` / `getAttribute`.

APPENDIX

---

# Sample Examination Solutions

---

Questions 4 & 5 — Applets, AWT, Events, Servlets, RMI & JDBC

# Security Restrictions on Java Applets

Applets run in a highly restricted browser-managed environment — the “sandbox” — designed to stop malicious code from harming the user's system. Primary restrictions:

## File System Access

Cannot read, write, rename, or delete files on the client's local file system.

## Network Access

May connect only back to the originating server host — never to arbitrary third-party servers.

## System Properties

Prevented from reading sensitive properties such as user account name or directory paths.

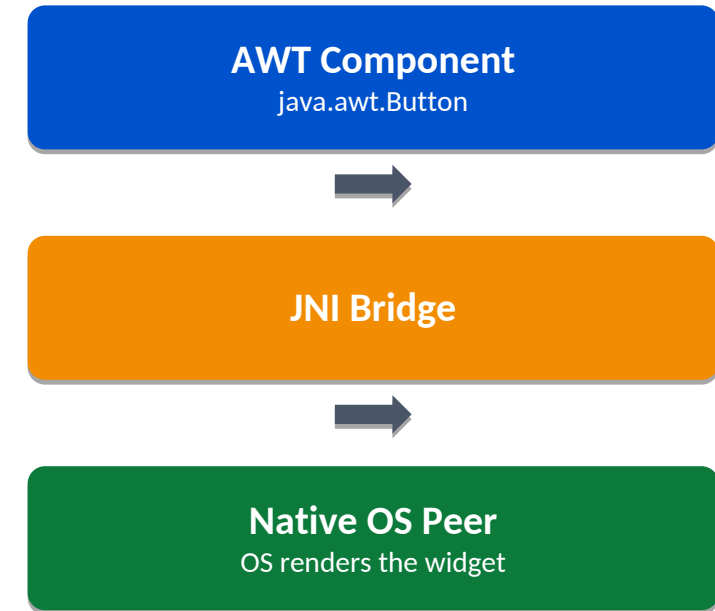
## Local Execution

Forbidden from starting local programs or executing native binaries on the client.

# Architecture of Java AWT

- ▶ **AWT** is Java's original platform-independent windowing, graphics & widget toolkit.
- ▶ Fundamentally “**heavyweight**”: every component (Button, TextField) has a native **peer** provided by the OS.
- ▶ Instantiating an AWT Button uses **JNI** to ask the OS (Windows / macOS / Linux) to render its own native button.
- ▶ **Result:** interfaces adopt the host OS look-and-feel automatically...
- ▶ ...but suffer **architectural rigidity** — the native OS controls behaviour & rendering sequence.

## HEAVYWEIGHT PEER MODEL



# Handling Events: The Delegation Event Model

1

## Identify the Event Source

The GUI component (e.g. a Button) that generates the event.

2

## Implement the Listener Interface

A class implementing the matching interface — e.g. ActionListener for clicks.

3

## Register the Listener

Link listener to source via a registration method — e.g. addActionListener().

4

## Define the Callback Logic

Write the code inside the overridden method — e.g. actionPerformed().

### ActionListener example

```
Button btn = new Button("Click Me");
btn.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        System.out.println("The button was successfully clicked!");
    }
});
```

# Event Handling — Complete Example

```
EventExample.java
import java.awt.*;
import java.awt.event.*;

public class EventExample {
    public static void main(String[] args) {
        Frame frame = new Frame("Event Example");
        Button btn = new Button("Click Me");

        // Step 2 & 3: implement and register the listener
        btn.addActionListener(new ActionListener() {
            // Step 4: define the callback logic
            public void actionPerformed(ActionEvent e) {
                System.out.println("The button was successfully clicked!");
            }
        });

        frame.add(btn);
        frame.setSize(200, 200);
        frame.setLayout(new FlowLayout());
        frame.setVisible(true);
    }
}
```

# Applet: Three Stacked Rectangles (Flag)

FlagApplet.java

```
import java.applet.Applet;
import java.awt.Color;
import java.awt.Graphics;

// Run via an Applet Viewer or an HTML wrapper
public class FlagApplet extends Applet {
    @Override
    public void paint(Graphics g) {
        g.setColor(Color.orange);           // top band
        g.fillRect(50, 50, 150, 40);

        g.setColor(Color.white);           // middle band
        g.fillRect(50, 90, 150, 40);

        g.setColor(Color.green);           // bottom band
        g.fillRect(50, 130, 150, 40);

        g.setColor(Color.black);           // outer border
        g.drawRect(50, 50, 150, 120);
    }
}
```

RENDERED OUTPUT



orange / white / green bands drawn with  
fillRect(), bordered by drawRect().

# Servlet Life Cycle & Instance Management

- ▶ Four deterministic phases, **managed entirely by the container (Tomcat)**:
  - **Loading & Instantiation** — loads the class, creates a single instance in memory.
  - **Initialization** — `init()` called once for bootstrapping (e.g. DB connections).
  - **Request Handling** — `service()` processes traffic, routing to `doGet()` / `doPost()`.
  - **Destruction** — `destroy()` called once to release memory & network resources.

## Instance Management

The container does NOT create a new Servlet per user. It keeps a single instance in memory and spawns multiple lightweight Java threads, routing each through the `service()` method of that one instance — ensuring high concurrency and performance.



# HTTP GET vs POST in Java Servlets

|                        |                                                 |                                                             |
|------------------------|-------------------------------------------------|-------------------------------------------------------------|
| <b>Purpose</b>         | doGet(): fetch / retrieve data from the server. | doPost(): send, update, or submit data to the server.       |
| <b>Data Visibility</b> | Parameters appended to & visible in the URL.    | Parameters hidden in the HTTP message body — safer.         |
| <b>Payload Size</b>    | Limited by max URL length (a few KB).           | Theoretically unlimited — needed for large forms / uploads. |
| <b>Caching</b>         | Can be cached, bookmarked & kept in history.    | Not bookmarkable; caching discouraged to avoid duplicates.  |

# Architecture of Java RMI

A multi-layered system letting an object in one JVM invoke methods on an object in a separate, remote JVM:

## 1 • Application Layer

The actual client and server code written by the developer.

## 2 • Proxy Layer (Stub & Skeleton)

Core mechanism: client-side Stub = local proxy; server-side Skeleton = receiver.

## 3 • Remote Reference Layer (RRL)

Manages the semantic object references across the network.

## 4 • Transport Layer

The underlying TCP/IP sockets that physically move the data.

### How communication flows

Client calls a method → Stub marshals (serializes) name & params into a byte stream → Transport Layer sends it → Skeleton unmarshals & invokes the real method → return value marshaled back to the Stub → unmarshaled & handed to the client.

# Architecture of JDBC

- ▶ **Two main components:**
  - **JDBC API** — used by the Java developer.
  - **JDBC Driver** — translates API calls into the DB's proprietary protocol.
- ▶ Deployed as **2-tier** (direct Java→DB) or **3-tier** (Java→app server→DB).

## How communication flows

DriverManager locates & loads the right driver (Type 1-4) from the connection string → Statement / PreparedStatement dispatch SQL → driver translates SQL to the DB's native language → results return in a ResultSet for seamless row-by-row iteration.

